

AWS Industrial Maintenance RAG Project

Step-by-Step Student Implementation Guide

What this guide helps you build

A complete serverless RAG application where manufacturing maintenance documents are uploaded to S3, converted to embeddings using Amazon Bedrock, stored in S3 Vectors, and queried through a browser UI hosted on S3.

Project component	AWS service / folder	Purpose
Document storage	Amazon S3 raw bucket	Stores original manufacturing maintenance text files.
Processed output	Amazon S3 processed bucket	Stores chunked JSON output after ingestion.
Vector database	Amazon S3 Vectors	Stores embeddings and supports similarity search.
Embedding model	Amazon Bedrock Titan Text Embeddings V2	Converts text into vectors.
Generation model	Amazon Bedrock Nova Lite	Generates final natural language answers.
Backend	AWS Lambda	Runs ingestion and query APIs.
Frontend	S3 static website	Browser UI for asking questions.

Recommended classroom duration: 2.5 to 3.5 hours

Contents

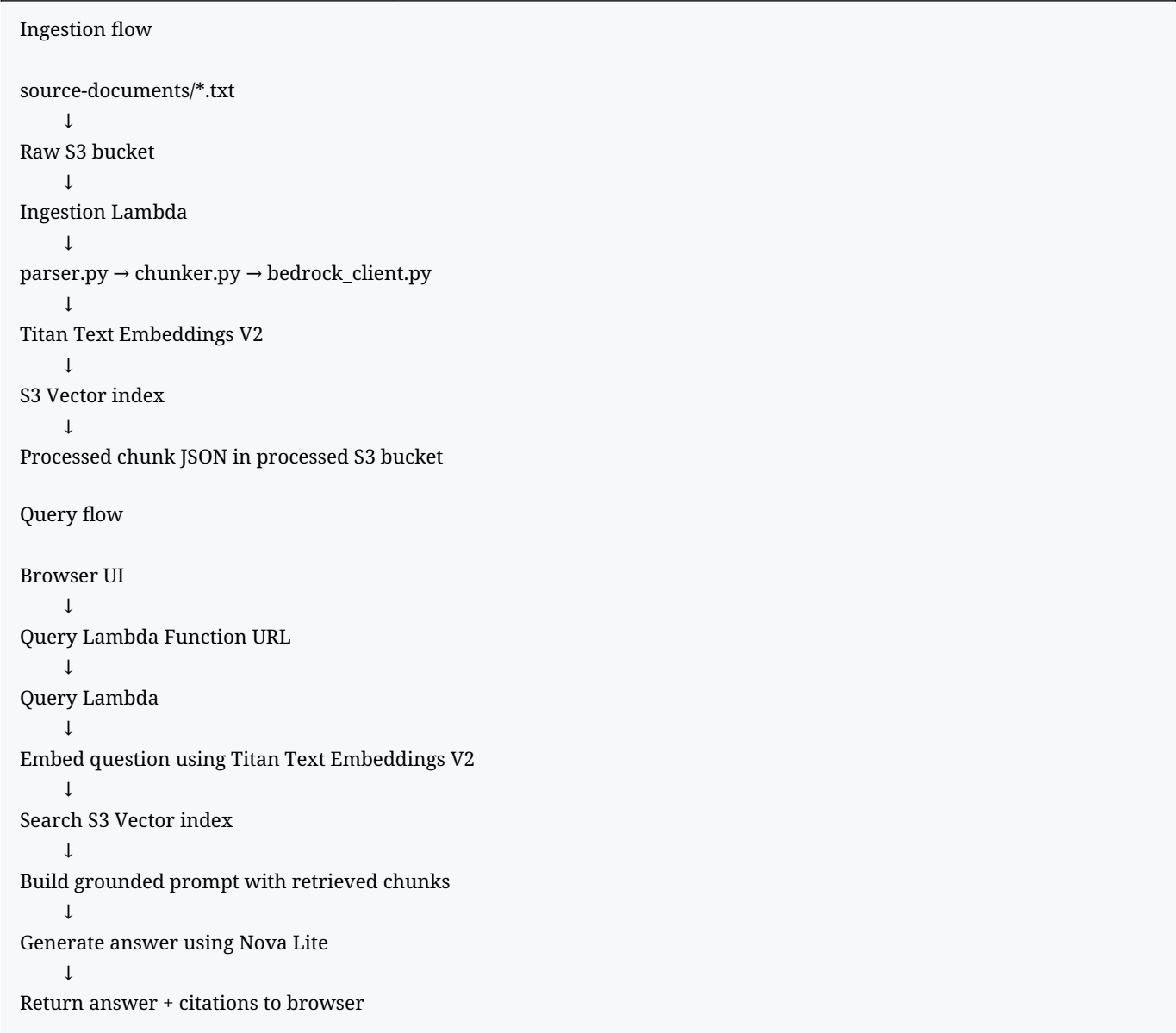
1. Project overview and architecture
2. Naming convention used in this guide
3. Prerequisites
4. Step 1 - Understand the project folder
5. Step 2 - Build Lambda zip files
6. Step 3 - Create normal S3 buckets
7. Step 4 - Create S3 Vector bucket and vector index
8. Step 5 - Enable Amazon Bedrock model access
9. Step 6 - Create IAM role and permissions
10. Step 7 - Deploy ingestion Lambda
11. Step 8 - Deploy query Lambda
12. Step 9 - Create Query Lambda Function URL
13. Step 10 - Configure frontend runtime config
14. Step 11 - Host frontend in S3
15. Step 12 - Upload source documents
16. Step 13 - Run ingestion Lambda
17. Step 14 - Test query Lambda directly
18. Step 15 - Test full frontend UI
19. Troubleshooting guide
20. File-by-file explanation
21. Classroom teaching script
22. Cleanup checklist and official references

1. Project overview and architecture

This project is a serverless Retrieval Augmented Generation application for a manufacturing maintenance knowledge base. Students upload maintenance documents, index them as vectors, and ask natural language questions from a browser UI.

The two major flows

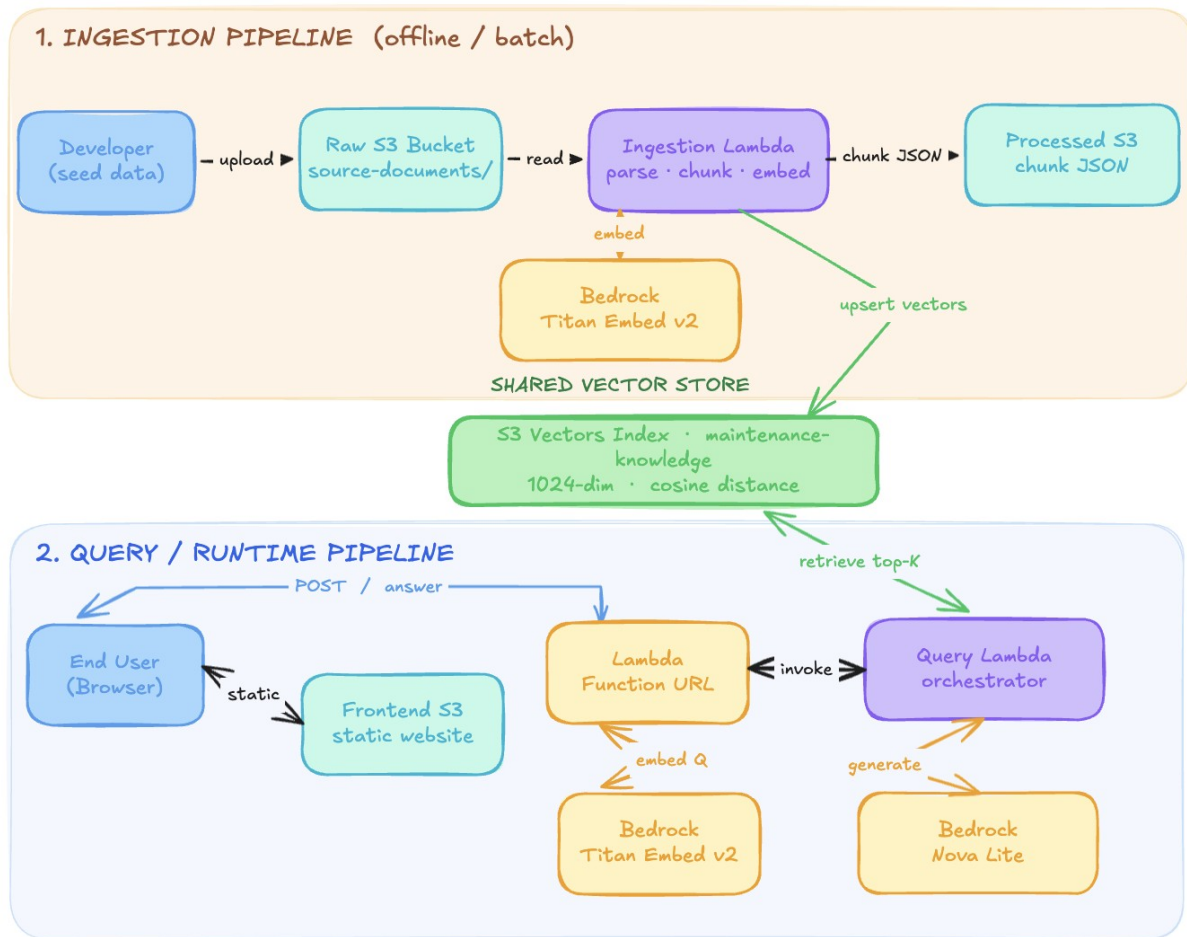
Flow	What happens	Main AWS services
Ingestion flow	Documents are parsed, chunked, embedded, and indexed.	S3, Lambda, Bedrock, S3 Vectors
Query flow	A question is embedded, similar chunks are retrieved, and a grounded answer is generated.	S3 website, Lambda Function URL, Bedrock, S3 Vectors



Architecture diagram from the project

Industrial Maintenance RAG on AWS

Serverless · Ingestion Pipeline + Query Pipeline



Key teaching line

RAG is not only asking an LLM. RAG is a data pipeline: documents -> chunks -> embeddings -> vector search -> prompt -> grounded answer with citations.

2. Naming convention used in this guide

For a classroom demo, use one consistent suffix for every AWS resource. In the live implementation, the suffix used was ss. Students can replace ss with their own initials or batch number.

Resource	Name used in this guide
Raw S3 bucket	industrial-maintenance-rag-manual-raw-ss
Processed S3 bucket	industrial-maintenance-rag-manual-processed-ss
Frontend S3 bucket	industrial-maintenance-rag-manual-frontend-ss
S3 Vector bucket	industrial-maintenance-rag-manual-vectors-ss
S3 Vector index	maintenance-knowledge
IAM role	industrial-maintenance-rag-manual-role-ss
Ingestion Lambda	industrial-maintenance-rag-ingestion-ss
Query Lambda	industrial-maintenance-rag-query-ss

Important

S3 bucket names are globally unique. If a bucket name is already taken, add your initials, city, or batch number.

3. Prerequisites

- AWS account with console access.
- Region selected as us-east-1 for this classroom implementation.
- Project folder extracted on laptop: rag_pipeline_manufacturing_industry.
- Python 3.10 or 3.11 installed.
- Git Bash recommended for Windows users.
- AWS CLI optional but useful.
- Bedrock model access available for Titan Text Embeddings V2 and Nova Lite.

4. Step 1 - Understand the project folder

```
rag_pipeline_manufacturing_industry/  
├── README.md  
├── Architecture.png  
├── frontend/  
│   ├── index.html  
│   ├── app.js  
│   ├── styles.css  
│   └── runtime-config.js  
└── scripts/
```

AWS Industrial Maintenance RAG - Student Implementation Guide

```
| |— build_lambda_packages.sh
| |— write_frontend_config.sh
|— sample_questions/
|   |— questions.json
|— source-documents/
|   |— sample maintenance .txt files
|— src/
|   |— common/
|   |— ingestion/
|   |— query_api/
|   |— data_generation/
```

Folder/File	Student explanation
frontend/	Static web UI used by students to ask questions.
source-documents/	Manufacturing maintenance documents that become the RAG knowledge base.
src/common/	Shared configuration, Bedrock client, S3 Vectors client, and JSON helpers.
src/ingestion/	Reads documents, parses metadata, chunks text, creates embeddings, and stores vectors.
src/query_api/	Receives questions, searches vector index, builds prompt, and returns answer.
scripts/	Helper scripts for packaging Lambdas and writing frontend config.

5. Step 2 - Build Lambda zip files

From the project root, try the script first in Git Bash:

```
cd ~/Desktop/rag_pipeline_manufacturing_industry
chmod +x scripts/build_lambda_packages.sh
./scripts/build_lambda_packages.sh
```

If chmod fails in PowerShell

chmod is a Linux or Mac command. If using Windows PowerShell, either open Git Bash or run the Python-based fallback below.

If Git Bash says zip: command not found

Use Python to create the zip files instead:

```
mkdir -p build

python - <<'PY'
```

AWS Industrial Maintenance RAG - Student Implementation Guide

```
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED

project_root = Path('.').resolve()
src_dir = project_root / 'src'
build_dir = project_root / 'build'
build_dir.mkdir(exist_ok=True)

def make_zip(zip_name):
    zip_path = build_dir / zip_name
    if zip_path.exists():
        zip_path.unlink()
    with ZipFile(zip_path, 'w', ZIP_DEFLATED) as z:
        for file_path in src_dir.rglob('*'):
            if file_path.is_file():
                z.write(file_path, file_path.relative_to(project_root))
    print(f'Created: {zip_path}')

make_zip('ingestion-lambda.zip')
make_zip('query-lambda.zip')
PY

ls -lh build
```

Expected output

build/ingestion-lambda.zip and build/query-lambda.zip. The zip must contain src/ at the root.

Correct zip structure

```
src/
  common/
  ingestion/
  query_api/
```

Wrong zip structure

```
rag_pipeline_manufacturing_industry/
  src/
```

6. Step 3 - Create normal S3 buckets

Create three normal S3 buckets in us-east-1.

Bucket	Public?	Purpose
industrial-maintenance-rag-manual-raw-ss	No	Original maintenance documents.
industrial-maintenance-rag-	No	Processed chunk JSON after

manual-processed-ss		ingestion.
industrial-maintenance-rag-manual-frontend-ss	Later yes	Static website files.

23. Open AWS Console -> S3 -> Create bucket.
24. Create raw bucket: industrial-maintenance-rag-manual-raw-ss.
25. Create processed bucket: industrial-maintenance-rag-manual-processed-ss.
26. Create frontend bucket: industrial-maintenance-rag-manual-frontend-ss.
27. Use region us-east-1.
28. Keep Block Public Access enabled for raw and processed buckets.
29. Keep frontend bucket private for now. We will make it public only during frontend hosting.

Optional AWS CLI

```
aws s3 mb s3://industrial-maintenance-rag-manual-raw-ss --region us-east-1
aws s3 mb s3://industrial-maintenance-rag-manual-processed-ss --region us-east-1
aws s3 mb s3://industrial-maintenance-rag-manual-frontend-ss --region us-east-1
```

7. Step 4 - Create S3 Vector bucket and vector index

S3 Vectors is different from normal S3 buckets. Normal buckets store files. Vector buckets store embeddings for similarity search.

30. Open S3 console.
31. In the left navigation, choose Vector buckets.
32. Create vector bucket: industrial-maintenance-rag-manual-vectors-ss.
33. Open the vector bucket.
34. Create vector index: maintenance-knowledge.

Index setting	Value
Vector index name	maintenance-knowledge
Data type	float32
Dimensions	1024
Distance metric	cosine

Why 1024 dimensions?

This project uses Amazon Titan Text Embeddings V2 with 1024 output dimensions. The vector index dimension must match the embedding dimension.

8. Step 5 - Enable Amazon Bedrock model access

- **Embedding model:** amazon.titan-embed-text-v2:0
- **Generation model:** amazon.nova-lite-v1:0

35. Open Amazon Bedrock in us-east-1.
36. Go to Model access or Model catalog, depending on your console layout.
37. Search for Titan Text Embeddings V2 and confirm access is available.
38. Search for Nova Lite and confirm access is available.
39. If you see Request access or Modify model access, enable access and wait a few minutes.

Classroom analogy

Titan converts text into meaning coordinates. S3 Vectors stores and searches those coordinates.
Nova Lite reads the retrieved notes and writes the final answer.

9. Step 6 - Create IAM role and permissions

Create one Lambda execution role shared by both Lambda functions.

40. Open IAM -> Roles -> Create role.
41. Trusted entity type: AWS service.
42. Use case: Lambda.
43. Attach AWSLambdaBasicExecutionRole.
44. Role name: industrial-maintenance-rag-manual-role-ss.
45. Create role.

Add inline policy

Open the role -> Permissions -> Add permissions -> Create inline policy -> JSON. Replace YOUR_ACCOUNT_ID with your 12-digit AWS account ID.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowRawBucketRead",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3:::industrial-maintenance-rag-manual-raw-ss",
        "arn:aws:s3:::industrial-maintenance-rag-manual-raw-ss/*"
      ]
    },
    {
      "Sid": "AllowProcessedBucketWrite",
      "Effect": "Allow",
```

AWS Industrial Maintenance RAG - Student Implementation Guide

```
"Action": ["s3:PutObject", "s3:GetObject", "s3:ListBucket"],
"Resource": [
  "arn:aws:s3:::industrial-maintenance-rag-manual-processed-ss",
  "arn:aws:s3:::industrial-maintenance-rag-manual-processed-ss/*"
],
},
{
  "Sid": "AllowBedrockModels",
  "Effect": "Allow",
  "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
  "Resource": [
    "arn:aws:bedrock:us-east-1::foundation-model/amazon.titan-embed-text-v2:0",
    "arn:aws:bedrock:us-east-1::foundation-model/amazon.nova-lite-v1:0"
  ]
},
{
  "Sid": "AllowS3VectorsAccess",
  "Effect": "Allow",
  "Action": [
    "s3vectors:GetVectorBucket",
    "s3vectors:ListIndexes",
    "s3vectors:GetIndex",
    "s3vectors:PutVectors",
    "s3vectors:GetVectors",
    "s3vectors:QueryVectors",
    "s3vectors:ListVectors"
  ],
  "Resource": [
    "arn:aws:s3vectors:us-east-1:YOUR_ACCOUNT_ID:bucket/industrial-maintenance-rag-manual-vectors-ss",
    "arn:aws:s3vectors:us-east-1:YOUR_ACCOUNT_ID:bucket/industrial-maintenance-rag-manual-vectors-ss/index/industrial-maintenance-knowledge"
  ]
}
]
```

Demo shortcut

For a fast classroom demo, AdministratorAccess can unblock permission issues. Explain clearly that production should use least privilege.

10. Step 7 - Deploy ingestion Lambda

46. Open Lambda -> Create function.
47. Choose Author from scratch.
48. Function name: industrial-maintenance-rag-ingestion-ss.
49. Runtime: Python 3.11.
50. Architecture: x86_64.

AWS Industrial Maintenance RAG - Student Implementation Guide

51. Use existing role: industrial-maintenance-rag-manual-role-ss.

52. Create function.

53. Code tab -> Upload from -> .zip file -> upload build/ingestion-lambda.zip.

Setting	Value
Handler	src.ingestion.handler.lambda_handler
Memory	1024 MB
Timeout	5 minutes

Environment variables for ingestion Lambda

```
RAW_BUCKET=industrial-maintenance-rag-manual-raw-ss
PROCESSED_BUCKET=industrial-maintenance-rag-manual-processed-ss
VECTOR_BUCKET_NAME=industrial-maintenance-rag-manual-vectors-ss
VECTOR_INDEX_NAME=maintenance-knowledge
VECTOR_DIMENSIONS=1024
EMBEDDING_MODEL_ID=amazon.titan-embed-text-v2:0
LOG_LEVEL=INFO
```

Test event to save for later

```
{
  "bucket": "industrial-maintenance-rag-manual-raw-ss",
  "prefix": "source-documents/"
}
```

Do not worry if a test now shows 0 objects

At this stage the documents may not be uploaded yet. We will run ingestion properly after Step 12.

11. Step 8 - Deploy query Lambda

54. Open Lambda -> Create function.

55. Choose Author from scratch.

56. Function name: industrial-maintenance-rag-query-ss.

57. Runtime: Python 3.11.

58. Architecture: x86_64.

59. Use existing role: industrial-maintenance-rag-manual-role-ss.

60. Create function.

61. Code tab -> Upload from -> .zip file -> upload build/query-lambda.zip.

Setting	Value
Handler	src.query_api.handler.lambda_handler
Memory	1024 MB

Timeout	1 minute
---------	----------

Environment variables for query Lambda

```
VECTOR_BUCKET_NAME=industrial-maintenance-rag-manual-vectors-ss
VECTOR_INDEX_NAME=maintenance-knowledge
VECTOR_DIMENSIONS=1024
EMBEDDING_MODEL_ID=amazon.titan-embed-text-v2:0
GENERATION_MODEL_ID=amazon.nova-lite-v1:0
LOG_LEVEL=INFO
```

Test event to save

```
{
  "question": "What should a technician inspect when filler FILL-203 reports intermittent pressure loss?",
  "filters": {
    "equipment_id": "FILL-203"
  },
  "top_k": 5
}
```

12. Step 9 - Create Query Lambda Function URL

62. Open Lambda -> industrial-maintenance-rag-query-ss.
63. Go to Configuration -> Function URL.
64. Choose Create function URL.
65. Auth type: NONE.
66. Enable CORS.
67. Allowed origin: * for initial testing.
68. Allowed methods: POST and OPTIONS.
69. Allowed headers: *.
70. Save and copy the generated Function URL.

Security note

Auth type NONE creates a public HTTPS endpoint. This is acceptable for a controlled classroom demo, but production should use stronger authentication.

Example Function URL format

```
https://abcxyz123.lambda-url.us-east-1.on.aws/
```

13. Step 10 - Configure frontend runtime config

The frontend reads the backend endpoint from frontend/runtime-config.js.

AWS Industrial Maintenance RAG - Student Implementation Guide

Use the helper script in Git Bash

```
./scripts/write_frontend_config.sh "https://abcxyz123.lambda-url.us-east-1.on.aws/"
```

Manual edit if script fails

```
window.RAG_CONFIG = {  
  apiEndpoint: "https://abcxyz123.lambda-url.us-east-1.on.aws/"  
};
```

Important

Use the Query Lambda Function URL, not the ingestion Lambda. Keep https:// and the final slash.

14. Step 11 - Host frontend in S3

71. Open S3 -> industrial-maintenance-rag-manual-frontend-ss.
72. Upload frontend/index.html, frontend/app.js, frontend/styles.css, and frontend/runtime-config.js directly into the bucket root.
73. Go to Properties -> Static website hosting -> Edit.
74. Enable static website hosting.
75. Index document: index.html.
76. Error document: index.html.
77. Save and copy the bucket website endpoint.

Optional AWS CLI upload

```
aws s3 sync frontend s3://industrial-maintenance-rag-manual-frontend-ss/ --delete
```

Make frontend bucket public for website access

78. Open frontend bucket -> Permissions.
79. Edit Block public access.
80. Uncheck Block all public access only for the frontend bucket.
81. Save changes and type confirm.
82. Add the bucket policy below.

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "PublicReadFrontendFiles",  
      "Effect": "Allow",  
      "Principal": "*",  
      "Action": "s3:GetObject",  
      "Resource": "arn:aws:s3:::industrial-maintenance-rag-manual-frontend-ss/*"
```

```
}  
]  
}
```

Correct frontend upload structure

index.html, app.js, styles.css, and runtime-config.js must be in the bucket root, not inside a frontend/ folder.

15. Step 12 - Upload source documents

Upload all .txt files from the local source-documents folder to the raw S3 bucket under the source-documents/ prefix.

Expected S3 structure

```
s3://industrial-maintenance-rag-manual-raw-ss/source-documents/  
ALARM-CODE-REFERENCE.txt  
INC-2026-0109-CONV-118.txt  
INC-2026-0142-FILL-203.txt  
MAN-CONV-118-BELT-TRACKING.txt  
PART-CASE-330-THERMISTOR.txt  
PART-VX-220-VALVE-ASSEMBLY.txt  
PM-FILL-203-AIR-PREP.txt  
PM-PUMP-707-SEAL.txt  
SOP-CASE-330-ADHESIVE-TEMP.txt  
SOP-FILL-203-PRESSURE-LOSS.txt  
SOP-PAL-410-LOCKOUT.txt  
SOP-VIS-520-REJECT-SPIKE.txt
```

Optional AWS CLI upload

```
aws s3 sync source-documents s3://industrial-maintenance-rag-manual-raw-ss/source-documents/ --delete  
aws s3 ls s3://industrial-maintenance-rag-manual-raw-ss/source-documents/
```

Prefix matters

The ingestion test event searches source-documents/. If files are uploaded to the bucket root, Lambda will see 0 objects.

16. Step 13 - Run ingestion Lambda

83. Open Lambda -> industrial-maintenance-rag-ingestion-ss.
84. Go to Test tab.
85. Create or select IngestionTest.
86. Use the JSON event below.
87. Click Test.

AWS Industrial Maintenance RAG - Student Implementation Guide

```
{
  "bucket": "industrial-maintenance-rag-manual-raw-ss",
  "prefix": "source-documents/"
}
```

Expected successful shape

```
{
  "status": "indexed",
  "objects_seen": 12,
  "documents": [
    {
      "document_id": "SOP-FILL-203-PRESSURE-LOSS",
      "chunk_count": 1,
      "vector_count": 1
    }
  ]
}
```

- Verify processed bucket has chunks or JSON output.
- Verify there are no AccessDenied, timeout, or s3vectors errors.
- If sample documents are small, most documents may create one chunk each.

17. Step 14 - Test query Lambda directly

This confirms the backend is working before testing the browser UI.

Lambda console test event

```
{
  "question": "What should a technician inspect when filler FILL-203 reports intermittent pressure loss?",
  "filters": {
    "equipment_id": "FILL-203"
  },
  "top_k": 5
}
```

Function URL curl test

```
curl -X POST "https://YOUR-FUNCTION-ID.lambda-url.us-east-1.on.aws/" -H "content-type: application/json" -d '{
  "question": "What should a technician inspect when filler FILL-203 reports intermittent pressure loss?",
  "filters": {"equipment_id": "FILL-203"},
  "top_k": 5
}'
```

Expected result

Status 200 with answer, citations, and retrieval details. If this works, Bedrock, Lambda, S3 Vectors, and IAM are working.

Additional test questions

No filter test

```
{
  "question": "What maintenance procedures are available in the knowledge base?",
  "filters": {},
  "top_k": 10
}
```

Incident report test

```
{
  "question": "What incident happened with FILL-203 and what was the corrective action?",
  "filters": {
    "equipment_id": "FILL-203",
    "document_type": "incident_report"
  },
  "top_k": 5
}
```

18. Step 15 - Test full frontend UI

88. Open S3 -> frontend bucket -> Properties -> Static website hosting.

89. Open the Bucket website endpoint in browser.

90. Ask a question using the UI.

Question:

What should a technician inspect when filler FILL-203 reports intermittent pressure loss?

Equipment:

FILL-203

Document type:

Any / blank

Top K:

5

Expected result

The page should show a generated answer and citations. This confirms the full end-to-end RAG application is working.

If the browser says Failed to fetch

If Step 14 works but the frontend says Failed to fetch, the backend is healthy. The problem is almost always frontend config, CORS, or browser caching.

AWS Industrial Maintenance RAG - Student Implementation Guide

91. Open the S3 website runtime-config.js URL and verify the Lambda Function URL.
92. Open Lambda -> query function -> Configuration -> Function URL -> Edit.
93. Set Allow origin to * for testing.
94. Set Allowed methods to POST and OPTIONS.
95. Set Allowed headers to *.
96. Save and hard refresh the browser using Ctrl + F5.

Open this in browser

<http://industrial-maintenance-rag-manual-frontend-ss.s3-website-us-east-1.amazonaws.com/runtime-config.js>

Expected content

```
window.RAG_CONFIG = {  
  apiEndpoint: "https://YOUR-ACTUAL-LAMBDA-FUNCTION-URL.lambda-url.us-east-1.on.aws/"  
};
```

What happened in the live implementation

The frontend started working after changing the Lambda Function URL CORS allowed origin to *. This proved that the issue was browser CORS, not Bedrock, Lambda code, IAM, or vector search.

19. Troubleshooting guide

Symptom	Likely cause	Fix
chmod not recognized	Running Linux command in PowerShell	Use Git Bash or Python zip fallback.
zip: command not found	Git Bash does not include zip utility	Use Python zip fallback from Step 2.
Unable to import module	Wrong zip structure or handler	Zip must contain src/ at root. Handler must be src.ingestion.handler.lambda_handler or src.query_api.handler.lambda_handler.
Missing environment variable	Lambda env vars incomplete	Recheck exact names and values.
objects_seen is 0	Documents not under source-documents/ prefix	Upload docs to raw bucket under source-documents/.
AccessDenied from S3	IAM role missing S3 permissions	Update inline policy for raw/processed bucket.
AccessDenied from Bedrock	Model access or IAM missing	Enable Bedrock access and

		bedrock:InvokeModel.
Unknown service s3vectors	Lambda boto3/botocore too old	Package latest boto3 and botocore with Lambda zip.
No matching knowledge	Vectors not indexed or filters too narrow	Run ingestion, check vector bucket/index names, try filters = {}.
Frontend 403	Frontend bucket public access/policy issue	Disable block public access only for frontend bucket and add bucket policy.
Frontend 404	index.html uploaded to wrong path	Upload frontend files to bucket root.
Failed to fetch	CORS or wrong runtime-config.js URL	Set Function URL CORS origin to * for testing and verify runtime-config.js.

Rebuild zips with latest boto3/botocore if needed

```
rm -rf build/package build/ingestion-lambda.zip build/query-lambda.zip
mkdir -p build/package

python -m pip install --upgrade boto3 botocore -t build/package
cp -r src build/package/

cd build/package
python - <<'PY'
from pathlib import Path
from zipfile import ZipFile, ZIP_DEFLATED

for name in ['./ingestion-lambda.zip', './query-lambda.zip']:
    zip_path = Path(name)
    if zip_path.exists():
        zip_path.unlink()
    with ZipFile(zip_path, 'w', ZIP_DEFLATED) as z:
        for p in Path('.').rglob('*'):
            if p.is_file():
                z.write(p, p)
    print(f'Created {zip_path}')
PY
cd ../..
```

20. File-by-file explanation

File	What it does
README.md	Project setup and deployment notes.

AWS Industrial Maintenance RAG - Student Implementation Guide

Architecture.png	Architecture diagram for teaching the flow.
scripts/build_lambda_packages.sh	Builds ingestion and query Lambda zip files.
scripts/write_frontend_config.sh	Writes Lambda Function URL into frontend/runtime-config.js.
frontend/index.html	Main browser page structure.
frontend/app.js	Reads input, calls query Lambda, renders answer and citations.
frontend/styles.css	Frontend styling.
frontend/runtime-config.js	Stores backend Query Lambda Function URL.
src/common/config.py	Reads environment variables.
src/common/bedrock_client.py	Calls Bedrock embedding and generation models.
src/common/s3_vectors_client.py	Wraps S3 Vectors put and query operations.
src/common/json_utils.py	Helpers for JSON serialization and Lambda responses.
src/ingestion/handler.py	Ingestion Lambda entry point. Lists S3 objects and sends each document to indexer.
src/ingestion/parser.py	Parses document front matter and body from .txt files.
src/ingestion/chunker.py	Splits document text into chunks and preserves metadata.
src/ingestion/indexer.py	Creates embeddings, writes vectors, and saves processed chunks.
src/query_api/handler.py	Query Lambda entry point. Embeds question, searches vectors, and generates answer.
src/query_api/prompt_builder.py	Builds grounded prompt with retrieved chunks and citation labels.
src/data_generation/generate_seed_data.py	Generates sample source documents and sample questions.
sample_questions/questions.json	Example questions for testing.
source-documents/*.txt	Manufacturing maintenance knowledge base.

21. Classroom teaching script

Opening explanation

Today we are building a real RAG application on AWS.

The LLM does not already know our factory maintenance documents.

First, we upload our documents. Then we convert them into embeddings.

When a user asks a question, we retrieve relevant chunks and give only those chunks to the LLM.

That is why the answer is grounded and can include citations.

Suggested delivery sequence

Time	Topic	Demo action
10 min	RAG concept	Explain documents -> chunks -> embeddings -> search -> answer.
10 min	Architecture	Show Architecture.png and two flows.
15 min	Code walkthrough	Explain src/common, src/ingestion, src/query_api, frontend.
30 min	AWS resources	Create S3, vector bucket, Bedrock access, IAM.
25 min	Lambda deployment	Deploy ingestion and query Lambdas.
20 min	Frontend	Configure Function URL and host frontend in S3.
20 min	End-to-end test	Upload docs, run ingestion, ask questions.
15 min	Troubleshooting	Show CORS, logs, and common errors.

Student explanation checkpoints

- Ask: Where are original documents stored? Answer: raw S3 bucket.
- Ask: Who converts text into vectors? Answer: Titan Text Embeddings V2 through Bedrock.
- Ask: Where are embeddings stored? Answer: S3 Vector index.
- Ask: Who creates the final answer? Answer: Nova Lite through Bedrock.
- Ask: Why do we need CORS? Answer: Browser frontend and Lambda URL are different origins.
- Ask: Why do citations matter? Answer: They show which retrieved chunks supported the answer.

22. Cleanup checklist and official references

Cleanup checklist

- Delete Lambda functions if no longer needed.
- Delete objects from S3 buckets before deleting buckets.
- Delete normal S3 buckets: raw, processed, frontend.
- Delete S3 Vector index and vector bucket.
- Delete IAM role and inline policy if created only for this demo.
- Review Bedrock usage and costs.

Official AWS references

- Lambda Function URLs - configuration: <https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html>
- Lambda Function URLs - auth model: <https://docs.aws.amazon.com/lambda/latest/dg/urls-auth.html>
- Amazon S3 static website hosting permissions: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteAccessPermissionsReqd.html>
- Amazon S3 static website tutorial: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/HostingWebsiteOnS3Setup.html>
- Amazon S3 Vectors overview: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors.html>
- Creating an S3 Vector index: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors-create-index.html>
- S3 Vectors index concepts: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/s3-vectors-indexes.html>
- Amazon Titan Text Embeddings models: <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>
- Amazon Titan Text Embeddings model parameters: <https://docs.aws.amazon.com/bedrock/latest/userguide/model-parameters-titan-embed-text.html>

Final memory hook

RAG = retrieve first, then generate. In this project, S3 + Lambda + Bedrock + S3 Vectors form a complete serverless RAG pipeline.